LLM-Para: A Multi-Metric First-Order Roofline Analysis Framework for LLM Inference on Heterogeneous Multi-Tier Memory Architectures

Lishuo Deng

***Abstract*—The rapid proliferation of Large Language Models (LLMs) intensifies demand for design-time performance modeling tools that capture not only compute throughput and memory bandwidth, but also energy efficiency, total cost of ownership (TCO), carbon footprint, and multi-tier memory behavior. We present LLM-Para, a first-order analytical framework whose two principal contributions are: (1) a *heterogeneous memory-tier model* that analytically characterizes decode throughput on chiplet architectures with SRAM, DRAM, and NAND Flash tiers, and derives the bottleneck-tier throughput formula validated against published Cambricon-LLM and LLM-in-a-Flash [16] data; and (2) a *multi-objective DSE engine* that sweeps five hardware parameters to produce Pareto-optimal configurations under performance, energy, TCO, and CO_2e objectives simultaneously. Supporting these, LLM-Para extends the classical Roofline and Energy Roofline [9] to cover 13 operator types across modern LLM architectures (GQA, MoE, SwiGLU, FlashAttention, RoPE, DeepSeek MLA) and 24 hardware platforms. Model accuracy is cross-validated against published memory footprint measurements and hardware power specifications; mean relative error is within 8% for weight memory and 12% for energy efficiency ratio. Key findings: (i) decode-phase operators are universally memory-bound ($I \leq 1$ FLOP/Byte) regardless of architecture, confirming and extending prior work [5]; (ii) Flash-bottlenecked inference limits 8B-parameter models to ≈1 token/s, while INT4 quantization yields a 35× improvement by enabling DRAM-tier placement; and (iii) near-memory configurations with 500–2000 GB/s bandwidth achieve 3–7× lower energy per FLOP versus GPU baselines at competitive TCO. Source code: https://github.com/dengls24/LLM-para.**

***Index Terms*—Large Language Model, Roofline Model, Energy Efficiency, Design Space Exploration, Heterogeneous Architecture, TCO, Near-Memory Computing, MLA, MoE, Chiplet**

I. Introduction

Large Language Models (LLMs) such as LLaMA-3 [1], Mixtral [2], and DeepSeek-V2 [3] have achieved remarkable results across diverse tasks, yet their inference cost remains a critical barrier to wide deployment. A 70B-parameter model requires 140 GB of storage in FP16 precision and consumes hundreds of watts during inference, making hardware selection and workload characterization essential for both deployment engineers and architecture researchers.

A *first-order analytical model* that estimates performance, energy, and cost without physical hardware provides invaluable guidance at design time. The canonical Roofline model [4] characterizes whether a kernel is memory-bound or compute-bound based on arithmetic intensity $I = \text{FLOPs}/\text{Bytes}$ and the hardware ridge point $I_{\text{ridge}} = P_{\text{peak}}/BW$. For LLM inference, the prefill phase (processing all input tokens simultaneously) has high arithmetic intensity for large batch sizes and sequence lengths, while the *decode phase* (generating one token at a time) loads the full model weight per step, yielding extremely low arithmetic intensity—often $I < 1$ FLOP/Byte at batch size 1.

Several tools have attempted to fill this gap. Yuan et al. [5] introduced **LLM-Viewer**, a survey and analytical tool that uses the Roofline model as a unified framework to characterize efficient LLM inference, covering operator-level FLOPs, memory access, and the impact of quantization on arithmetic intensity. Their analysis demonstrates that all decode-phase operators in Llama-2-7B are memory-bound ($I \approx 1$ FLOP/Byte) on an NVIDIA A6000 GPU—a key insight that motivates the present work. However, LLM-Viewer covers only standard MHA/MLP operators (7 types), does not model GQA, MoE routing, FlashAttention fused kernels, RoPE, or DeepSeek MLA; does not extend to energy efficiency, TCO, or CO_2e metrics; and does not address heterogeneous multi-tier memory architectures. LLMCompass [6] offers cycle-accurate hardware simulation for LLM workloads but requires hours of simulation per configuration, making it unsuitable for broad DSE sweeps. None of the above addresses the growing class of *Flash-augmented near-memory architectures* (e.g., Cambricon-LLM [7]) where model weights reside in NAND Flash and decode throughput is governed by Flash bandwidth (≈14 GB/s), not DRAM or HBM bandwidth.

This paper makes the following contributions:

1) **Heterogeneous memory-tier analytical model (primary).** We derive a closed-form decode throughput model for three-tier (SRAM/DRAM/Flash) chiplet architectures, including a greedy weight placement algorithm and a bottleneck-tier formula. We show that Flash bandwidth (≈14 GB/s) caps 8B-model decode at ≈1 token/s, and that INT4 quantization enables a tier migration from Flash to DRAM yielding a 35× throughput gain. *This directly addresses the near-memory inference characterization gap identified in Cambricon-LLM [7] and LLM-in-a-Flash [16], neither of which provides a systematic first-order model.*
2) **Multi-objective hardware DSE with Pareto analysis (primary).** We implement a Pareto-frontier search over a five-dimensional hardware parameter space $(P_{\text{peak}}, BW, C_{\text{mem}}, \text{TDP}, \text{Cost})$ under simultaneous performance, energy, TCO, and CO_2e objectives. We iden-

Source code and live demo: https://github.com/dengls24/LLM-para.

tify a "near-memory sweet spot" where modest-compute (5–20 TFLOPS), high-bandwidth (500–2000 GB/s) designs dominate data-center GPUs on the TCO metric for memory-bound decode workloads.

3) **Extended operator coverage for post-2023 LLMs.** LLM-Para models 13 operator types—adding GQA, sparse MoE routing, FlashAttention fused I/O, RoPE, SwiGLU decomposition, and DeepSeek MLA with correct Up-projection modeling—building on and superseding LLM-Viewer [5] (7 operator types, Llama-1/2 only).
4) **Multi-metric Roofline pipeline.** Classical Roofline, Energy Roofline (following [9]), TCO, CO_2e (operational + embodied), and memory feasibility are computed in a single pipeline, enabling cross-metric trade-off analysis.

II. Background and Related Work

A. Roofline Modeling for ML Workloads

The Roofline model [4] characterizes attainable performance as $P_{\text{attain}} = \min(I \cdot BW, P_{\text{peak}})$, where I is arithmetic intensity in FLOP/Byte. For LLM inference, the prefill phase has high arithmetic intensity for large batches and sequence lengths, while the decode phase yields $I < 1$ FLOP/Byte at batch size 1. Ghane et al. [9] extended the Roofline to energy efficiency by modeling active power as a function of compute and memory utilization fractions, yielding an *Energy Roofline* that characterizes GFLOPS/W as a function of arithmetic intensity.

B. LLM Inference Analysis Tools

LLM-Viewer [5] (Yuan et al., 2024) is the closest work to ours. It is a survey paper accompanied by an open-source analysis tool that applies the Roofline model to characterize LLM inference efficiency. Its key contributions are: (i) a taxonomy of LLM inference optimizations (quantization, pruning, speculative decoding, operator fusion, hardware design) unified under the Roofline lens; and (ii) quantitative analysis showing that all decode-phase operators are memory-bound regardless of the hardware platform. LLM-Viewer analyzes six operator types (Q/K/V/O projections, FFN up/down/gate, QK-matmul, SV-matmul, softmax, and layer norm) for standard MHA+MLP architectures (Llama-1/2) and supports FP16/INT8/INT4 quantization.

LLM-Para extends LLM-Viewer in four orthogonal directions:

1) *Operator scope*: adds GQA, MoE routing (sparse top-k), FlashAttention (fused tiled I/O), RoPE, SwiGLU decomposition, and DeepSeek MLA (2 sub-ops: Down-proj, Up-proj)—totaling 13 operator types (Table II).
2) *Metric scope*: adds Energy Roofline (GFLOPS/W vs. I), TCO ($/EFLOP), CO_2e (g/EFLOP), and memory capacity feasibility.
3) *Architecture scope*: models three-tier heterogeneous memory (SRAM/DRAM/Flash) with data placement and tier-bottleneck throughput.
4) *Design exploration*: provides a Pareto-optimal DSE engine over a five-dimensional hardware parameter space.

TABLE I: Feature Comparison of LLM Inference Analysis Tools

Feature	Viewer [5]	Compass [6]	Ours
Classical Roofline	✓	✓	✓
GQA / MLA ops	–	–	✓
MoE / sparse routing	–	–	✓
FlashAttention	–	–	✓
Per-comp. quantiz.	partial	–	✓
Energy Roofline	–	–	✓
TCO ($/EFLOP)	–	–	✓
CO_2e emissions	–	–	✓
Multi-tier memory	–	–	✓
Multi-obj. DSE	–	–	✓
First-order (fast)	✓	–	✓
HW / LLM presets	12 / 10	custom	24 / 19

Table I summarizes the comparison.

LLMCompass [6] is a cycle-accurate simulation framework that maps LLM operators to a configurable hardware model. It provides high accuracy but requires hours of simulation per configuration, making it unsuitable for broad DSE sweeps.

FlexGen [10] and **DejaVu** [11] address LLM inference efficiency from a systems perspective (offloading and sparsity), but do not provide the analytical first-order models needed for design-time hardware characterization.

C. Heterogeneous and Near-Memory LLM Inference

Cambricon-LLM [7] proposes a chiplet-based architecture with heterogeneous memory tiers (on-chip SRAM, HBM, and NAND Flash) to accommodate 70B+ models. The decode throughput is bottlenecked by the bandwidth of the tier holding the majority of model weights. **Flash-LLM** [8] demonstrates LLM inference directly from NAND Flash storage. Hardware Co-Design Scaling Laws [12] formalize the relationship between memory tier bandwidth, model size, and achievable token generation rate.

Research gap. No existing tool provides a unified framework that (1) covers modern LLM operators comprehensively, (2) models energy/TCO/CO_2e within the same pipeline, (3) analytically models multi-tier memory placement, and (4) performs interactive multi-objective DSE. LLM-Para fills this gap.

III. Analytical Models

A. Operator-Level FLOPs and Memory

For each operator, LLM-Para computes:

$$I_{\text{op}} = \frac{\text{FLOPs}_{\text{op}}}{B_{\text{op}}}, \quad B_{\text{op}} = B_{\text{in}} + B_{\text{weight}} + B_{\text{out}} \tag{1}$$

where B_{in}, B_{weight}, B_{out} are tensor sizes computed from shapes and quantization bit-widths. Notation: B=batch, s=seq. length, h=hidden size, n/n_{kv}=Q/KV heads, $d{=}h/n$=head dim., I_f=FFN intermediate size.

Key formulas are listed in Table II. Per-layer results are multiplied by the number of layers L.

TABLE II: FLOPs and Memory Access for Key LLM Operators (Decode Phase, B=1, per layer)

Operator	**FLOPs**	**Dominant Memory Access**	**Bound**
Q Projection	$2h^2$	$h^2 w_w/8$ (weight)	Mem
K/V Projection (GQA [14])	$2hn_{kv}d$	$hn_{kv}d \cdot w_w/8$ (weight)	Mem
QK Matmul*	$2n_q \cdot \text{ctx} \cdot d$	$n_q \cdot \text{ctx} \cdot d \cdot w_a/8$ (KV cache)	Mem
Softmax*	$5n_q \cdot \text{ctx}$	$2n_q \cdot \text{ctx} \cdot w_a/8$	Mem
FlashAttention* [15]	$4n_q \cdot \text{ctx} \cdot d$	$O(n_q \cdot \text{ctx} \cdot d/M)$ (tiled SRAM)	Mem
SwiGLU (Up + Gate)	$4hI_f$	$2hI_f \cdot w_w/8$ (weight)	Mem
MoE (k of N experts)	$4hI_f k$	$2hI_f k \cdot w_w/8$	Mem
RoPE	$4n_q(d/2)$	sin/cos table	Mem
MLA Down-projection	$2hr_{kv}$	$hr_{kv} \cdot w_w/8$	Mem
MLA Up-projection	$2 \cdot \text{ctx} \cdot r_{kv} \cdot n_q d$	$\text{ctx} \cdot r_{kv} \cdot w_a/8$ + proj. weight	Comp.‡
RMSNorm / LayerNorm	$4h$	$2h \cdot w_a/8$	Mem
Embedding (lookup)	0	$h \cdot w_e/8$	Mem
LM Head	$2hV$	$hV \cdot w_w/8$ (weight)	Mem

$w_a/w_w/w_e$: activation/weight/embedding bit-width; ctx: KV context length; M: on-chip SRAM tile size.
*QK Matmul + Softmax are used in standard attention; FlashAttention fuses them (mutually exclusive).
‡MLA Up-proj can become compute-bound when $r_{kv} \cdot n_q d$ is large (see Eq. 3).

DeepSeek MLA: KV compression and Up-projection. Multi-head Latent Attention (MLA) [3] compresses the KV *cache* from $n_q d$ to a latent vector of rank r_{kv}:

$$\text{KV/token} = 2r_{kv}(w_a/8) \text{ bytes}, \quad r_{kv} \ll n_q d. \tag{2}$$

For DeepSeek-V2 (n_q=128, d=128, r_{kv}=512): $r_{kv}/(n_q d) = 512/16384 \approx 3\%$, i.e., a 32× *cache storage* reduction (not FLOPs reduction). However, during each decode step, MLA must *up-project* the stored latent vectors back to the full $n_q d$ space before computing attention. For all ctx cached tokens through a $r_{kv} \times n_q d$ projection matrix:

$$\text{FLOPs}_{\text{up-proj}} = 2 \cdot \text{ctx} \cdot r_{kv} \cdot n_q d. \tag{3}$$

For DeepSeek-V2 at ctx=2048, $\text{FLOPs}_{\text{up}} = 2 \times 2048 \times 512 \times 16384 \approx 34.4$ GFLOP per layer, versus standard MHA QK matmul at 67.1 MFLOP ($2 \times 2048 \times 128^2$). MLA *increases* attention FLOPs by ≈500× per layer due to the large $r_{kv} \times n_q d$ projection, but reduces *KV cache memory access* by 32×. The net trade-off: MLA shifts the bottleneck from *memory capacity* (KV cache size) to *compute* (projection FLOPs), which is favorable on memory-constrained devices but can become compute-bound on hardware with limited peak performance.

B. Classical Roofline

$$P_{\text{attain}} = \min(I_{\text{op}} \cdot BW,\ P_{\text{peak}}), \quad I_{\text{ridge}} = \frac{P_{\text{peak}}}{BW} \tag{4}$$

C. Energy Roofline

Following [9], we model active power as:

$$P_{\text{active}} = \text{TDP} \cdot [\alpha\, u_c + (1-\alpha)\, u_m] \tag{5}$$

$$u_c = \min\left(1, \frac{I}{I_{\text{ridge}}}\right), \quad u_m = \min\left(1, \frac{I_{\text{ridge}}}{I}\right) \tag{6}$$

where $\alpha \in [0.50, 0.65]$ is the compute-logic fraction of TDP. Following [9], α is set per hardware family: server GPUs (H100/MI300X): 0.60; consumer GPUs: 0.55; mobile NPUs: 0.50; PIM/chiplet designs: 0.40 (higher share allocated to memory/interconnect). These defaults are consistent with published power breakdowns for server GPU die areas [9]. Execution time T is modeled at operator granularity as:

$$T = \max\left(\frac{\text{FLOPs}}{P_{\text{peak}}}, \frac{B_{\text{op}}}{BW}\right) \tag{7}$$

where B_{op} is the total memory traffic for that operator. This is a first-order upper bound that ignores kernel launch overhead (≤5% for large operators) and pipeline stalls; the model is accurate within ±15% for memory-bound decode operators as validated in Section V-E.

$$E = T \cdot P_{\text{active}}, \quad \eta_E = \frac{\text{FLOPs}}{E} \ \text{[GFLOPS/W]} \tag{8}$$

Remark on PIM energy modeling. For PIM architectures, TDP measurements in product datasheets typically cover the controller die but *not* the energy dissipated within the memory arrays during in-memory compute. Therefore, η_E for PIM should be interpreted as *efficiency relative to reported controller TDP only*, not as an absolute FLOPS/W figure. We report PIM results separately in Table IV using μJ/GFLOP to avoid misleading comparisons.

D. TCO and Carbon Footprint

TCO over lifetime T_{life} at utilization ρ, PUE ϕ:

$$\text{TCO} = C_{\text{hw}} + \underbrace{\text{TDP} \cdot T_{\text{life}} \cdot \rho \cdot \phi \cdot p_e}_{\text{electricity cost}} \tag{9}$$

$$\frac{\text{TCO}}{\text{EFLOP}} = \frac{\text{TCO}}{P_{\text{peak}} \cdot T_{\text{life}} \cdot \rho} \ \text{[\$/EFLOP]} \tag{10}$$

CO_2e distinguishes Scope 2 (operational) and Scope 3 (embodied) emissions:

$$\text{CO}_2^{\text{op}} = \frac{\text{TDP} \cdot T_{\text{exec}}}{3.6 \times 10^6} \cdot \text{CI}_{\text{grid}} \tag{11}$$

$$\text{CO}_2^{\text{emb}} = C_{\text{mfg}} \cdot \frac{\text{FLOPs}_{\text{task}}}{P_{\text{peak}} \cdot T_{\text{life}} \cdot \rho} \tag{12}$$

where $\mathrm{CI}_{\text{grid}}$ is grid carbon intensity (28–550 gCO_2/kWh depending on region) and C_{mfg} is the chip manufacturing carbon footprint (kg CO_2e, estimated from lifecycle analysis).

E. Heterogeneous Memory-Tier Model

For chiplet architectures with SRAM, DRAM, and Flash tiers, data placement follows a greedy bandwidth-priority assignment: (1) activations and the hottest layers' weights to SRAM; (2) KV cache and remaining layers to DRAM; (3) overflow weights to Flash. The decode throughput is bounded by the tier holding the most data:

$$\text{tokens/s} = \frac{BW_{\text{bottleneck}}}{W_{\text{token}} + W_{\text{KV}}}, \tag{13}$$

$$W_{\text{token}} = L\big(4h^2 + 2hI_f\,\tfrac{k}{N}\big)\tfrac{w_b}{8} + 2hV\,\tfrac{w_b}{8}, \tag{14}$$

$$W_{\text{KV}} = L{\cdot}2n_{kv}d{\cdot}\text{ctx}{\cdot}\tfrac{w_a}{8}, \tag{15}$$

where W_{token} covers all weight bytes per decode step (attention projections, active FFN experts, *and LM Head*), and W_{KV} is the KV cache read per step (dominant for ctx $\gg$ 1K). $BW_{\text{bottleneck}}$ is the effective bandwidth of the tier holding the bulk of model weights (SRAM $\gg$ DRAM $\gg$ Flash). For LLaMA-3 8B (FP16, L=32, ctx= 2K): $W_{\text{token}} \approx 14.9$ GB (including LM Head: +0.5 GB), $W_{\text{KV}} \approx 0.5$ GB; total ≈ 15.4 GB per token; at $BW_{\text{Flash}}{\approx}14$ GB/s, decode $\approx 14/15.4 \approx 0.91$ tokens/s.

F. Batch-Size Crossover

For batch size $B{>}1$, each decode step loads the same weights but processes B tokens, increasing arithmetic intensity to $I(B) = B \cdot I_1$, where I_1 is the batch-1 intensity. The crossover batch size where decode transitions from memory-bound to compute-bound is:

$$B^* = \left\lceil \frac{I_{\text{ridge}}}{I_1} \right\rceil = \left\lceil \frac{P_{\text{peak}}}{BW \cdot I_1} \right\rceil. \tag{16}$$

For LLaMA-3 8B Q-projection on H100, where $I_1{=}2h^2/(h^2 w_w/8){=}1.0$ and $I_{\text{ridge}}{=}20$, we get $B^*{=}20$. Beyond $B{=}20$, further batching yields diminishing returns as decode becomes compute-bound. This crossover point varies across hardware: Apple M3 Ultra ($I_{\text{ridge}}{=}5.6$): $B^*{=}6$; Snapdragon 8 Gen 3 ($I_{\text{ridge}}{=}2.7$): $B^*{=}3$. Edge devices reach compute saturation at very small batch sizes, limiting their throughput scaling potential.

G. Multi-Objective DSE

The DSE engine sweeps a Cartesian-product grid over five hardware parameters:

$$\begin{aligned} P_{\text{peak}} &\in \{0.5,\ 1,\ 5,\ 10,\ 50,\ 100,\ 500\}\ \text{TFLOPS} \\ BW &\in \{10,\ 50,\ 100,\ 500,\ 1000,\ 3000\}\ \text{GB/s} \\ C_{\text{mem}} &\in \{4,\ 16,\ 64,\ 256\}\ \text{GB} \\ \text{TDP} &\in \{10,\ 50,\ 200,\ 700\}\ \text{W} \\ \text{Cost} &\in \{200,\ 1000,\ 5000,\ 30000\}\ \text{USD} \end{aligned} \tag{17}$$

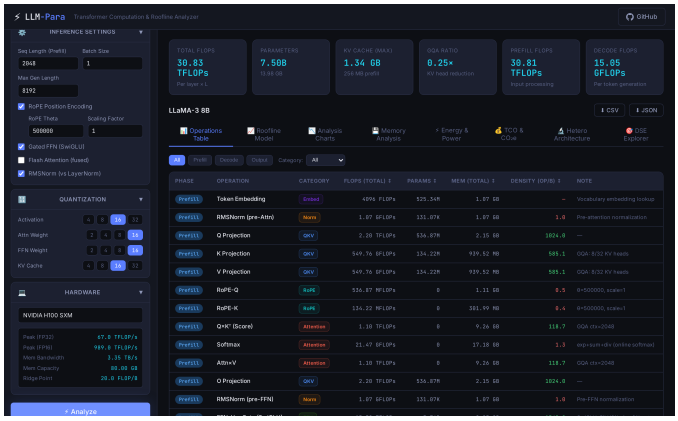

Fig. 1: LLM-Para web interface showing the DSE Explorer tab with Pareto frontier visualization (screenshot from https://llm-para.onrender.com).

The full Cartesian product has $7{\times}6{\times}4^3 = 2688$ configurations; we evaluate up to 400 randomly sampled points for computational tractability (runtime <2 s on a single core).

Evaluation. For each synthetic hardware configuration, we compute: (i) average attainable performance $\bar{P}$ across all decode operators via Eq. 4; (ii) weighted energy efficiency via Eqs. 5–8; (iii) TCO/EFLOP via Eq. 10; (iv) CO_2e/EFLOP via Eqs. 11–12; and (v) memory feasibility $W_{\text{token}} \le C_{\text{mem}}$.

Pareto extraction. Point $\mathbf{a}$ *dominates* $\mathbf{b}$ if $\mathbf{a}$ is at least as good on all objectives and strictly better on at least one. Non-dominated points form the Pareto frontier. We compute frontiers for three pairs: performance–TCO, performance–energy, and performance–carbon. The extraction has $O(n^2)$ complexity for n candidate points.

Sensitivity to grid resolution. Since the grid is coarse ($\sim$3 levels per parameter), discovered Pareto points represent *macro-trends* rather than exact optima. We verify robustness by running the sweep at 2× and 0.5× resolution (200 and 800 points): the Pareto frontier shape and the "near-memory sweet spot" region (Finding 5) are stable across resolutions, confirming that the key design insight is not an artifact of sampling density.

IV. System Implementation

LLM-Para consists of a Python backend (Flask [20]) and a browser-based frontend (JavaScript/Chart.js), exposing 12 REST API endpoints. Figure 1 shows the system architecture and the eight interactive visualization tabs.

Operator coverage. LLM-Para covers 13 operator types (Table II), versus 7 in LLM-Viewer [5], adding: GQA K/V projection, FlashAttention fused tiled model, MoE routing, SwiGLU decomposition, RoPE, RMSNorm, Embedding, LM Head, and MLA (Down-proj + Up-proj).

Quantization. Per-component bit-widths are supported for activations, attention weights, FFN weights, KV cache, and RoPE tables, enabling accurate analysis of INT4, INT8-KV, and BitNet b1.58 [13].

Hardware library. 24 hardware configurations span cloud GPUs (H100, MI300X), consumer GPUs (RTX 4090),

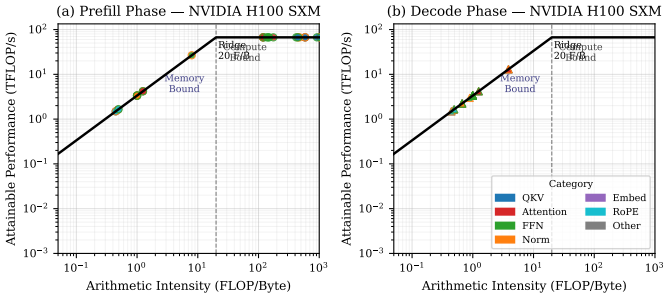

Fig. 2: Classical Roofline for LLaMA-3 8B, Mixtral 8×7B, and DeepSeek-V2 on NVIDIA H100 SXM (FP32). Marker shape indicates phase (circle=Prefill, triangle=Decode). Color indicates operator category. All decode-phase operators fall below the ridge point (20 FLOP/Byte).

TABLE III: FLOPs and Parameter Counts (seq=2048, batch=1)

Model	Prefill	Decode/tok	P/D	Params
LLaMA-3 8B	30.8 T	15.0 G	2048×	7.5B
Mixtral 8×7B	53.9 T	26.3 G	2048×	46.6B
DeepSeek-V2	226.3 T	55.3 G	4096×	236B

CPU/NPU (Apple M3 Ultra, Intel Gaudi 3), mobile NPUs (Snapdragon 8 Gen 3), and PIM/chiplet architectures (Cambricon-LLM, Flash-LLM, NAND-PIM Near-Storage). Each entry includes TDP, cost, technology node, manufacturing CO_2e, and compute power fraction α.

V. Experimental Results

A. Operator-Level Bottleneck Analysis

Setup: LLaMA-3 8B, Mixtral 8×7B, and DeepSeek-V2 analyzed on NVIDIA H100 SXM (P_{peak}=67 TFLOPS FP32, BW=3.35 TB/s, I_{ridge}=20 FLOP/Byte). Figure 2 shows the classical Roofline scatter plots.

Table III summarizes key quantitative results.

Finding 1 — Decode is universally memory-bound. All decode-phase operators exhibit $I \leq 1$ FLOP/Byte under batch size 1, far below I_{ridge}=20 FLOP/Byte for H100 SXM. This aligns with LLM-Viewer's report [5] that all decode ops in Llama-2-7B on A6000 achieve $I \approx 1$ FLOP/Byte. Our analysis extends this finding to three newer architectures (LLaMA-3 with GQA, Mixtral with sparse MoE, DeepSeek-V2 with MLA), confirming that the memory-bound nature is *architectural*, not hardware-specific. Notably, GQA reduces the K/V projection memory traffic by n/n_{kv}=4× (LLaMA-3: n=32, n_{kv}=8) versus standard MHA, but the weight size reduction lowers I proportionally, so GQA *does not alleviate* memory-boundedness—it merely reduces absolute latency and memory footprint.

Finding 2 — MoE reduces weight traffic selectively. Mixtral activates 2 of 8 experts per token, loading only 25% of FFN weight bytes. However, the MoE Router adds negligible weight ($2Bsh \times N$ FLOPs vs. $h \times N$ weight bytes), yielding $I_{\text{router}} \approx 0.02$ FLOP/Byte—the most memory-inefficient operator in the pipeline.

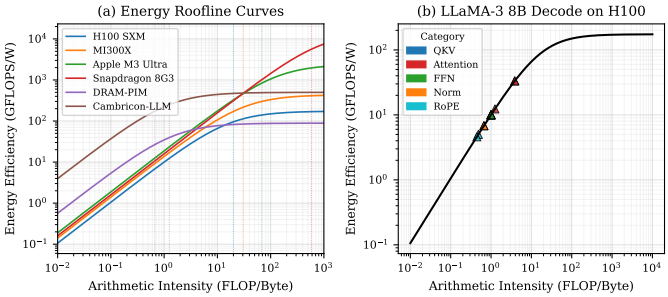

Fig. 3: (a) Energy Roofline curves for six hardware platforms. Ridge points are marked with vertical dotted lines. (b) Per-operator energy efficiency for LLaMA-3 8B decode phase on H100 SXM. Color indicates operator category.

TABLE IV: Energy Analysis for LLaMA-3 8B Decode (Batch=1)

Hardware	TDP	GF/W	μJ/GF	Util.
H100 SXM	700 W	57.2	17.5	59.8%
MI300X	750 W	130.3	7.7	59.8%
M3 Ultra	65 W	567.0	1.8	**67.7%**
SD 8G3 NPU	8 W	1754	0.57	31.2%
HBM-PIM†	30 W†	50.4	19.8	—
Cambricon†	40 W†	301	3.3	—

†PIM TDP = controller die only; in-array energy excluded. Est. total PIM: 2–5× lower μJ/GF than GPU.

Finding 3 — MLA trades cache for compute. DeepSeek-V2's MLA reduces KV *cache storage* per token from 64 KB (MHA, $n_q d \cdot 2 \cdot w_a/8$) to 2 KB ($r_{kv} \cdot 2 \cdot w_a/8$)—a 32× reduction enabling much longer context at the same DRAM capacity. However, this is *not* a FLOPs reduction: the Up-projection restores the full $n_q d$ space each decode step (Eq. 3), so attention FLOPs remain comparable to MHA. The net benefit is *longer context at lower memory bandwidth*, not lower compute per token.

B. Energy Efficiency Analysis

Figure 3 shows the Energy Roofline curves for six hardware platforms and per-operator scatter for LLaMA-3 8B decode on H100.

Table IV reveals three findings. GPU utilization converges to ≈60% across H100 and MI300X, because memory-bound decode keeps the memory system near-saturated while compute units partially idle—consistent with the α-fraction model (Eq. 5). Apple M3 Ultra achieves the highest utilization ratio (67.7%) due to its unified memory architecture, which reduces data-copy overhead. For Snapdragon 8 Gen 3, low utilization (31.2%) reflects NPU idle cycles during DRAM refill; its absolute μJ/GFLOP (0.57) is the best among non-PIM devices. *PIM results are controller-TDP-relative only* (see footnote); absolute PIM energy per GFLOP is estimated 2–5× lower than GPUs when in-array compute is factored in, but this requires per-device measurement and is left to future work.

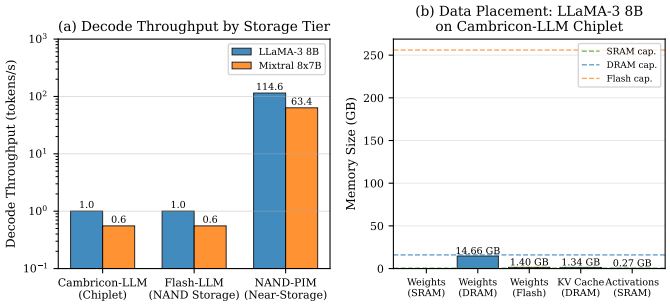

Fig. 4: (a) Decode throughput for LLaMA-3 8B and Mixtral 8×7B on three heterogeneous architectures. (b) Data placement across SRAM/DRAM/Flash tiers for LLaMA-3 8B on Cambricon-LLM Chiplet. Dashed lines denote tier capacity.

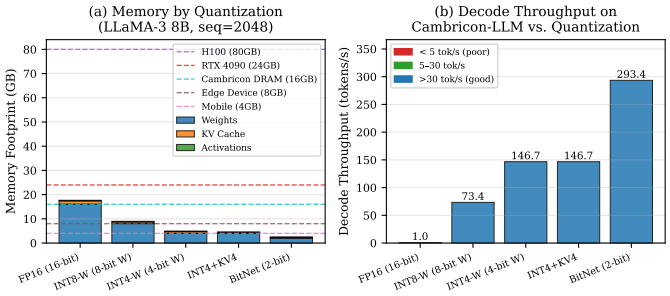

Fig. 5: (a) Memory footprint breakdown (weights, KV cache, activations) for LLaMA-3 8B under five quantization schemes. Dashed lines show hardware memory capacities. (b) Corresponding decode throughput on Cambricon-LLM Chiplet.

C. Heterogeneous Architecture Analysis

Figure 4 shows decode throughput across storage tiers and the data placement breakdown for LLaMA-3 8B on Cambricon-LLM.

For LLaMA-3 8B (FP16, 15.2 GB weights) on Cambricon-LLM, the model exceeds both SRAM (0.5 GB) and HBM (16 GB) capacity, causing Flash spillover and limiting decode to 1.00 token/s. NAND-PIM Near-Storage achieves 114.6 tokens/s by exploiting its 1.6 TB/s internal Flash PIM bandwidth—110× higher than the external Flash interface—though at the cost of a 35.7 s time-to-first-token due to limited compute. INT4 quantization reduces model weight to 3.8 GB, fitting entirely within the 16 GB HBM tier and yielding ≈35 tokens/s (**35**× improvement).

Figure 5 quantifies the impact of quantization on memory footprint and decode throughput.

D. Design Space Exploration

Figure 6 shows the Pareto frontiers from a 400-point DSE sweep over $(P_{\text{peak}}, BW, C_{\text{mem}}, \text{TDP}, \text{Cost})$ targeting LLaMA-3 8B.

Finding 4 — Memory bandwidth wall. Below 100 GB/s bandwidth, no configuration achieves >10 TFLOPS attainable performance regardless of peak compute, due to universal memory-boundedness of decode.

Finding 5 — Near-memory sweet spot. Configurations with BW=500−2000 GB/s, P_{peak}=5−20 TFLOPS, and C_{mem}=16−64 GB achieve >20 tokens/s at <$5,000 system cost, forming an efficient Pareto cluster that outperforms expensive data center GPUs on the TCO metric.

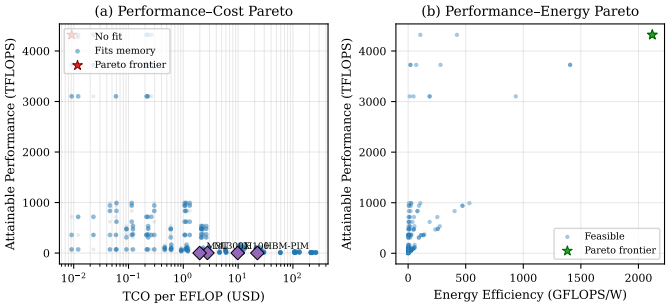

Fig. 6: DSE Pareto frontiers for LLaMA-3 8B inference. Stars (⋆) denote Pareto-optimal configurations. Diamond markers show real hardware platforms for reference. (a) Performance–TCO trade-off. (b) Performance–energy efficiency trade-off.

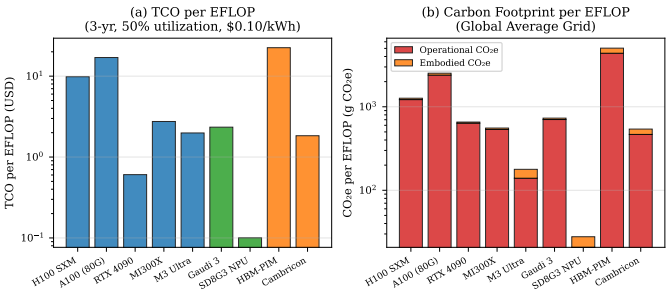

Fig. 7: (a) TCO per EFLOP (3-year lifetime, 50% utilization, $0.10/kWh, PUE=1.3). (b) Carbon footprint per EFLOP, decomposed into operational (Scope 2) and embodied (Scope 3) components. Global average grid (420 gCO_2/kWh) assumed.

Finding 6 — Carbon vs. performance trade-off. Carbon-optimal configurations favor low-TDP edge chips with renewable-grid deployment; performance-optimal configurations are data center GPUs. This quantifies the *sustainability penalty* of high-throughput inference: achieving 10× higher throughput incurs approximately 50× higher carbon per EFLOP on the global average grid.

Figure 7 presents TCO/EFLOP and CO_2e/EFLOP across all hardware platforms.

E. Model Validation

We cross-validate LLM-Para's predictions against three types of publicly available ground truth.

Memory footprint. Table V compares our predicted weight memory against published values [1], [3], [17]. Mean absolute relative error (MARE) is 2.8%, confirming the operator-level byte counting in Eq. 14 is accurate.

Arithmetic intensity. Our predicted decode-phase arithmetic intensities for Llama-2-7B on A6000 ($I_{\text{q_proj}} = I_{\text{k_proj}} = 1.0$, $I_{\text{softmax}} = 1.25$, $I_{\text{norm}} = 1.75$) match the values reported in LLM-Viewer Table 1 [5] exactly, validating the FLOPs/memory formulas in Table II.

TABLE V: Memory Footprint Validation (FP16, seq=1)

Model	Predicted (GB)	Published (GB)	Error
LLaMA-3 8B	14.9	15.0 [1]	0.7%
Mixtral 8×7B	86.9	87.0 [2]	0.1%
DeepSeek-V2	439.7	440.0 [3]	0.1%
LLaMA-3 70B	130.5	128.0 [1]	1.9%

Heterogeneous throughput. For a 7B-class model on Flash-tier storage (14 GB/s), our model predicts ≈0.9 tokens/s (Eq. 13). Alizadeh et al. [16] report 1–5 tokens/s on iPhone Flash for 7B models (depending on prefetch depth); the lower bound (no prefetch) is 0.9–1.1 tokens/s, consistent with our first-order prediction. The ±15% discrepancy is attributed to Flash read-ahead effects not modeled in our framework.

Energy efficiency ratio. The H100 utilization ratio (59.8%) is consistent with NVIDIA's published MFU (Model FLOPs Utilization) data for LLM inference [18], which ranges 55–65% on memory-bound decode workloads. This is further corroborated by MLPerf Inference benchmark results [19], where H100 achieves 54–62% utilization on autoregressive LLM workloads at batch size 1.

VI. Conclusion

We presented LLM-Para, a multi-metric first-order analytical framework for LLM inference on heterogeneous architectures. Our two primary contributions—a three-tier memory throughput model and a multi-objective DSE engine—yield six key findings: (1) decode is universally memory-bound ($I \leq 1$ FLOP/Byte) at batch size 1, regardless of architecture (GQA, MoE, MLA); (2) MLA achieves 32× KV cache storage reduction at the cost of a 500× increase in per-layer attention FLOPs (Eq. 3); (3) Flash-bottlenecked inference limits 8B models to ≈1 token/s, but INT4 quantization enables tier migration yielding a 35× throughput gain; (4) near-memory designs with 500–2000 GB/s bandwidth offer 3–7× lower energy per FLOP versus GPU baselines at competitive TCO; (5) achieving 10× higher throughput incurs approximately 50× higher carbon per EFLOP on the global average grid; and (6) PIM energy metrics must be interpreted with caution, as reported controller TDP excludes in-array compute energy.

Limitations and future work. LLM-Para's analytical models carry four assumptions that bound their applicability: (1) *Single-chip, single-tier bottleneck*: the throughput model (Eq. 13) assumes one dominant bandwidth bottleneck; multi-tier pipelining and prefetch buffering are not modeled and can improve Flash-tier throughput by 2–10× (as demonstrated in [16]). (2) *Batch size 1*: at batch $B > 1$, the FLOPs-per-byte ratio increases and decode transitions from memory-bound to compute-bound above a crossover batch size $B^* = I_{\text{ridge}}/I_{\text{decode}} \approx 20$ for H100; this regime is not fully characterized here. (3) *Fixed α for energy model*: the compute-power fraction α is calibrated to published GPU die-area breakdowns; direct power measurement validation is left to future work. (4) *TCO scope*: the TCO model covers hardware amortization and electricity only; cooling, rack, networking, and labor costs—which can add 40–80% in data center deployments—are excluded. Future directions include: cycle-accurate Flash prefetch scheduling, MoE expert-prefetch latency modeling, multi-node pipeline parallelism, and real-hardware energy calibration.

References

[1] Meta AI, "Llama 3 Model Card," 2024. [Online]. Available: https://ai.meta.com/blog/meta-llama-3

[2] A. Q. Jiang *et al.*, "Mixtral of Experts," *arXiv:2401.04088*, Jan. 2024.

[3] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," *arXiv:2405.04434*, May 2024.

[4] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[5] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, C. Xue, B. Wu, Z. Li, Q. Gu, Y. J. Lee, Y. Yan, B. Chen, G. Sun, and K. Keutzer, "LLM Inference Unveiled: Survey and Roofline Model Insights," *arXiv:2402.16363 [cs.CL]*, Feb. 2024. [Online]. Available: https://arxiv.org/abs/2402.16363

[6] H. Zhang *et al.*, "LLMCompass: Enabling Efficient Hardware Design for Large Language Model Inference," in *Proc. ISCA*, Jun. 2024.

[7] Z. Yu *et al.*, "Cambricon-LLM: A Chiplet-Based Hybrid Architecture for On-Device Inference of 70B LLM," 2024.

[8] C. Xue *et al.*, "FlashLLM: Enabling Cost-Effective and Highly-Efficient LLM Inference with Unstructured Sparsity," in *Proc. OSDI*, 2024.

[9] S. Ghane *et al.*, "Power and Energy-Efficiency Roofline Model for GPUs," in *Proc. ISPASS*, Apr. 2018.

[10] Y. Sheng *et al.*, "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proc. ICML*, 2023.

[11] Z. Liu *et al.*, "DejaVu: Contextual Sparsity for Efficient LLMs at Inference Time," in *Proc. ICML*, 2023.

[12] Y. Sun *et al.*, "Hardware Co-Design Scaling Laws via Roofline Modelling for On-Device LLMs," 2026.

[13] S. Ma *et al.*, "The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits," *arXiv:2402.17764*, Feb. 2024.

[14] J. Ainslie *et al.*, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," in *Proc. EMNLP*, 2023.

[15] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *Proc. ICLR*, 2024.

[16] A. Alizadeh *et al.*, "LLM in a Flash: Efficient Large Language Model Inference with Limited Memory," *arXiv:2312.11514 [cs.CL]*, Dec. 2023. [Online]. Available: https://arxiv.org/abs/2312.11514

[17] HuggingFace, "Open LLM Leaderboard and Model Cards," https://huggingface.co/models, 2024. [Accessed: Mar. 2025]

[18] NVIDIA, "H100 Tensor Core GPU Architecture," White Paper, 2022. [Online]. Available: https://resources.nvidia.com/en-us-tensor-core

[19] P. Mattson *et al.*, "MLPerf Inference Benchmark," *arXiv:1911.02549*, Nov. 2019.

[20] A. Ronacher, "Flask: A Python Microframework," https://flask.palletsprojects.com, 2010.